```
Array
(
    [captain] => Sisko
    [medic] => Bashir
)
```

`unset()` does its job and destroys the value at index `0` in `$staff`. Note that the element has been completely removed from the array, and it now contains just two. You might also have noticed the keys have not been reindexed: Bob and Russell are still indexed with 1 and 2. The `$crew` array is associative, so we use one of the keys to unset `$crew['engineer']` (poor Myles can't catch a break).

If the disruption of sequential indexing is for any reason problematic to your program, it is possible to remove array elements and update the indexes by using (you guessed it) more of PHP's convenient built-in array functions.

Reindexing Elements

When an indexed array is first created, the indexes are sequential, and the first index value is `0`. If elements are removed, the sequence can be disrupted:

```
$myArr1 = ["Alice", "Bob", "Russell"];
$myArr2 = ["Alice", "Bob", "Russell"];

unset($myArr1[1]);
print_r($myArr1);

$myArr1 = array_values($myArr1);
print_r($myArr1);

array_splice($myArr2, 0, 1);
print_r($myArr2);

Output:
Array
(
    [0] => Alice
    [2] => Russell
)
```